

Archivos

Programación I

UNRN

Universidad Nacional
de Río Negro

17

Setiembr

e

Programación I

Clase 12: Diccionarios. Registros. Escritura en archivos.

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

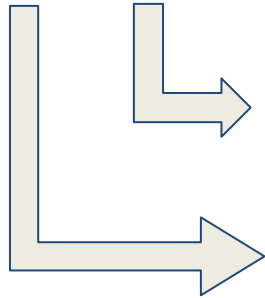
Sede Bariloche

Contenidos de la clase

- Archivos
- Diccionarios.
- Registros.
- Escritura en archivos (binarios y de texto).

01 Archivos

FILE* fopen(char* *nombre_archivo*, char* **modo**);  Si falla, apuntará a NULL, en caso correcto devuelve FILE*



nombre_archivo: Es una cadena de caracteres (char*) que indica la ruta o el nombre del archivo que deseas abrir (por ejemplo, "datos.txt").

modo: Es una cadena de caracteres (char*) que define cómo se va a utilizar el archivo.

- "r" (Read): Abre un archivo existente **“sólo”** para lectura. Si el archivo no existe, falla.
- "w" (Write): Abre un archivo para escritura. Si el archivo no existe, lo crea. Si ya existe, borra todo su contenido anterior.
- "a" (Append): Abre un archivo para agregar datos al final. Si no existe, lo crea. Los datos nuevos no borran los anteriores, sino que se escriben al final.
- Si además se agrega una + (como "r+", "w+", "a+") permite combinar lectura y escritura.
- Agregando una b (como "rb", "wb") indica que el archivo se tratará en modo binario en lugar de texto.

02 Archivos: veamos un ejemplo

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    // 1. Crear o sobrescribir un archivo para escritura
    FILE *archivo_escritura = fopen("ejemplo.txt", "w");

    // 2. Validar si el archivo se abrió correctamente
    if (archivo_escritura == NULL) {
        printf("Error al crear el archivo.\n");
        return 1; // Terminar el programa con código de error
    }

    // 3. Escribir texto en el archivo
    fprintf(archivo_escritura, "Buenas! Este es un ejemplo de fopen en C.\n");
    fprintf(archivo_escritura, "Línea número 2.\n");

    // 4. Cerrar el archivo (siempre es obligatorio cerrarlo)
    fclose(archivo_escritura); // esta función devuelve un entero
    printf("Archivo escrito con éxito.\n\n");
}
```

```
// 1. Abrir el archivo existente solo para lectura
FILE *archivo_lectura = fopen("ejemplo.txt", "r");
```

```
// Validar si el archivo existe y se puede leer
if (archivo_lectura == NULL) {
    printf("Error al abrir el archivo para lectura.\n");
    return 1;
}
```

```
// 2. Leer el contenido del archivo línea por línea y mostrarlo en pantalla
```

```
char buffer[100];
printf("Contenido del archivo:\n");
while (fgets(buffer, 100, archivo_lectura) != NULL) {
    printf("%s", buffer);
}
```

```
// 3. Cerrar el archivo de lectura
fclose(archivo_lectura);
```

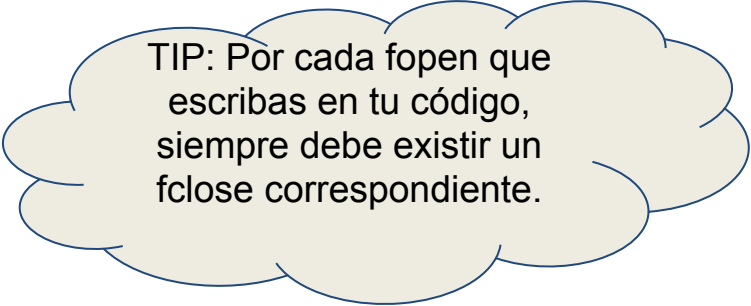
```
return 0;
}
```

Retorna 0: si el archivo se cerró con éxito.
Retorna EOF (un valor negativo, usualmente -1) si ocurrió un error al intentar cerrarlo.

03 Archivos: verificación de fclose

```
#include <stdio.h>
```

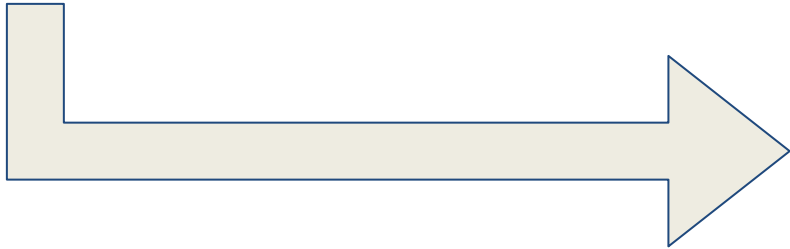
```
int main() {  
    // 1. Abrimos el archivo  
    FILE *archivo = fopen("ejemplo.txt", "w");  
  
    if (archivo == NULL) {  
        printf("Error al abrir el archivo.\n");  
        return 1;  
    }  
  
    // 2. Operamos con el archivo  
    fprintf(archivo, "Hola, mundo. Guardando datos en C.\n");  
  
    // 3. Cerramos el archivo y verificamos si se cerró bien  
    if (fclose(archivo) == 0) {  
        printf("Archivo cerrado con éxito y datos guardados.\n");  
    } else {  
        printf("Error al cerrar el archivo.\n");  
    }  
  
    return 0;  
}
```



TIP: Por cada fopen que escribas en tu código, siempre debe existir un fclose correspondiente.

04 Archivos: Escritura a archivo (2 formas)

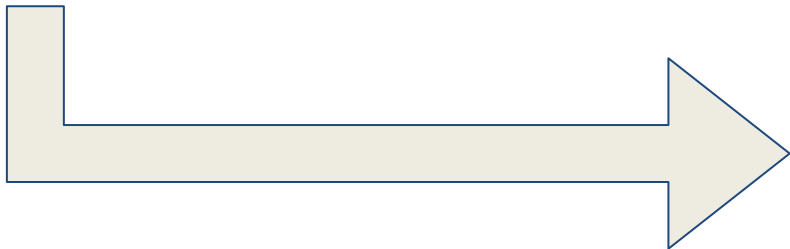
int fprintf(FILE* stream, ...);



```
#include <stdio.h>

FILE *archivo;
archivo = fopen("salida.txt", "w"); // deberíamos chequear
fprintf(archivo, "Hola, mundo!\n");
fclose(archivo);
```

int fputc(int ch, FILE* stream);



```
FILE *archivo;
char texto[] = "Escribiendo caracter por caracter.";
int i = 0;
archivo = fopen("salida_fputc.txt", "w");
while (texto[i] != '\0') {
    fputc(texto[i], archivo);
    i++;
}
fputc('\n', archivo);
fclose(archivo);
```

04 Archivos: Escritura a archivo (2 formas.. y pros/contras)

Característica	<code>fprintf</code>	<code>fputc</code>
Propósito	Escribe cadenas de texto con formato.	Escribe un único carácter.
Argumentos	Puntero al archivo, cadena de formato, variables.	El carácter (como <code>int</code>), puntero al archivo.
Flexibilidad	Alta (convierte números, una textos, etc.). <code>fprintf(archivo, "El usuario %s tiene %d años\n", nombre, edad);</code>	Baja (solo un byte/carácter).
Velocidad	Más lenta (requiere procesar la cadena).	Muy rápida (operación directa).
Consumo de memoria	Mayor (carga funciones de conversión).	Mínimo.

04 Archivos: Lectura de archivo

Carácter a carácter: (int fgetc(FILE *stream);)

```
FILE* archivo = fopen("salida_fputc.txt", "r");
int caracter;

caracter = fgetc(archivo);
while (!feof(archivo)) {
    putchar(caracter);
    caracter = fgetc(archivo);
}
fclose(archivo);
```

1 dale que va
0
0 no hay más archivo

Leer de a bloques: (int fgetc(FILE *stream);)

```
#define MAX_CADENA 10

char bloque[MAX_CADENA];
FILE* archivo = fopen("salida_fputc.txt", "r");
while (fgets(bloque, sizeof(bloque), archivo) != NULL)
{ printf("%s\n", bloque); }

fclose(archivo);
```

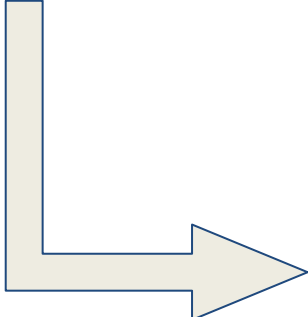
str
0
NULL

En el ejemplo se
repetiria hasta feof

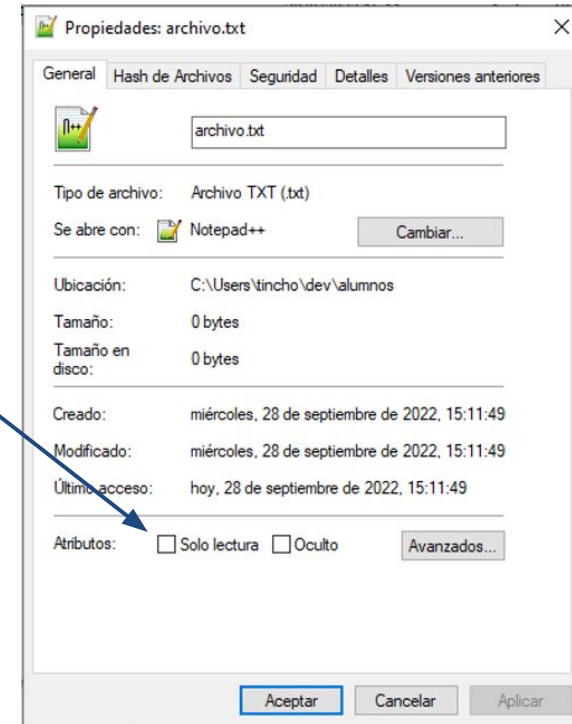
04 Archivos: Ejercicio

- 1) Escribir un programa en C que solicite al usuario ingresar el nombre y el precio de 3 productos. El programa debe crear (o sobrescribir) un archivo llamado productos.txt y guardar la información ingresada utilizando la función fprintf, **dándole un formato de lista**.
- 2) Crear un programa en C que genere un archivo de texto llamado abecedario.txt. Utilizando un bucle for y la función fputc, el programa debe escribir todas las letras mayúsculas del abecedario inglés (de la 'A' a la 'Z') en el archivo, separadas por un espacio o un guión.

04 Archivos: Gestión de errores

- 
- El archivo no existe
 - Permisos de archivo
 - Archivo abierto

`a1 = fopen("archivo.txt", "r");`



04 Archivos: Gestión de errores, como saber que fallo?

perror en stdio.h

```
#include <stdio.h>
```

```
int main() {
```

```
    // Intentamos abrir un archivo que no existe en modo lectura ("r")
```

```
    FILE *archivo = fopen("archivo_inexistente.txt", "r");
```

```
    if (archivo == NULL) {
```

```
        // La función falló, así que usamos perror para saber por qué
```

```
        perror("Fallo al intentar abrir el archivo");
```

```
        return 1;
```

```
    }
```

```
    fclose(archivo);
```

```
    return 0;
```

```
}
```



Fallo al intentar abrir el archivo: ***No
such file or directory***

04 Archivos: Gestión de errores, como saber que fallo?

```
#include <stdio.h>
#include <string.h> // Obligatoria para usar la función strerror()
#include <errno.h> // Obligatoria para acceder a la variable global 'errno'
```

```
int main() {
```

```
    // Intentamos abrir un archivo que no existe
    FILE *archivo = fopen("no_existi.txt", "r");
```

```
    if (archivo == NULL) {
        // Método 1: Traducción manual del error
        printf("%s\n\n", strerror(errno));
```

tomará ese dos, buscará en su diccionario interno y devolverá el texto "No such file or directory".

```
        // Método 2: El atajo automático
        perror("fopen() fallo");
    }
```

```
    else {
        // todo OK
        printf("El archivo se abrió correctamente.\n");
```

```
        // Siempre es buena práctica cerrar el archivo si se logró abrir
        fclose(archivo);
    }
```

```
    return 0;
}
```

<https://man7.org/linux/man-pages/man3/errno.3.html>

04 Archivos: Gestión de errores, como saber que fallo?

```
#include <stdio.h>
#include <string.h> // Obligatoria para usar la función strerror()
#include <errno.h> // Obligatoria para acceder a la variable global 'errno'
```

```
int main() {
```

```
    // Intentamos abrir un archivo que no existe
    FILE *archivo = fopen("no_existi.txt", "r");
```

```
    if (archivo == NULL) {
        // Método 1: Traducción manual del error
        printf("%s\n\n", strerror(errno));
```

tomará ese dos, buscará en su diccionario interno y devolverá el texto "No such file or directory".

```
        // Método 2: El atajo automático
        perror("fopen() fallo");
    }
```

```
    else {
        // todo OK
        printf("El archivo se abrió correctamente.\n");
```

```
        // Siempre es buena práctica cerrar el archivo si se logró abrir
        fclose(archivo);
    }
```

```
    return 0;
}
```

<https://man7.org/linux/man-pages/man3/errno.3.html>

04 Archivos: Gestión de errores, como saber que fallo?

```
#include <stdio.h>
#include <string.h> // Obligatorio para usar strerror()
#include <errno.h> // Obligatorio para acceder a 'errno'
```

```
int main() {
    // Intentamos abrir un archivo que no existe en el disco
    FILE *archivo = fopen("archivo fantasma.txt", "r");

    // Validamos si la operación falló
    if (archivo == NULL) {
        // 1. Capturamos el número del error que el sistema guardó en la sombra
        int codigo_error = errno;

        // 2. La función nos devuelve la dirección de memoria donde está el texto traducido
        char *mensaje_traducido = strerror(codigo_error);

        // 3. Imprimimos el resultado usando %d para el número y %s para el texto
        printf("Fallo la operacion.\n");
        printf("Codigo numerico del sistema: %d\n", codigo_error);
        printf("Explicacion del error: %s\n", mensaje_traducido);

    } else {
        printf("Archivo abierto exitosamente.\n");
        fclose(archivo);
    }

    return 0;
}
```

Retorna un puntero a la cadena que describe el error indicado por errno

04 Archivos: escritura binaria

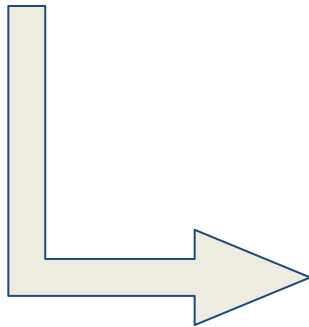
fopen() usando el modo "wb" para escribir o "rb" para leer (la 'b' es por binario).

fwrite() que copia un bloque de memoria directamente al disco.

fread() que toma los bytes del disco y los mete en nuestras variables. Y siempre,

fclose() para cerrar el archivo de forma segura.

- La principal ventaja es el rendimiento: son muchísimo más rápidos porque la computadora no tiene que traducir los datos a texto.
- Son ideales para guardar estructuras completas (structs) de un solo golpe.
- Además, ocupan menos espacio en disco y ofrecen una capa básica de privacidad, ya que no se pueden leer a simple vista.



En resumen, si vas a guardar un texto simple, usa archivos de texto.

Pero si estás programando un videojuego, una base de datos o un sistema que maneja **muchos datos complejos**, los archivos binarios son la mejor opción en C.

04 Archivos: escritura binaria

```
#include <stdio.h>
```

```
typedef struct {  
    int id;  
    float calificacion;  
} Estudiante;
```

```
int main() {  
    Estudiante e1 = {101, 9.5};  
    Estudiante e2; // Aquí guardaremos lo que leamos
```

```
    // 1. ESCRIBIR EN EL ARCHIVO  
    FILE *archivoW = fopen("datos.bin", "wb");  
    if (archivoW != NULL) {  
        fwrite(&e1, sizeof(Estudiante), 1, archivoW);  
        fclose(archivoW);  
        printf("Datos guardados exitosamente.\n");  
    }
```

```
    // 2. LEER DEL ARCHIVO
```

```
    FILE *archivoR = fopen("datos.bin", "rb");  
    if (archivoR != NULL) {  
        fread(&e2, sizeof(Estudiante), 1, archivoR);  
        fclose(archivoR);
```

```
        // Mostrar resultado en pantalla  
        printf("Datos leidos -> ID: %d, Nota: %.1f\n",  
            e2.id, e2.calificacion);  
    }  
    return 0;  
}
```